

Cấu trúc dữ liệu và thuật toán

Assignment 4

Giảng viên: Nguyễn Thị Tâm

§ Cây nhị phân - cây tổng quát §

Phần 1: Mục tiêu

- Nắm được các khái niệm liên quan đến cây tổng quát, cây nhị phân.
- Giải thích được các cách biểu diễn cây, và có khả năng vẽ cây từ một biểu diễn cây và ngược lại.
- Giải thích và triển khai được thuật toán chuyển đổi cây tổng quát thành cây nhị phân và ngược lại
- Giải thích và triển khai được cách biểu diễn cây nhị phân trên máy tính
- Phát triển thuật toán và triển khai được thuật toán cho cây tổng quát và cây nhị phân.

Phần 2: Bài tập thực hành

- Các tính chất của cây nhị phân

1. Tính chiều cao của cây nhị phân - [Height of Binary Tree](#)

– Ref - <https://www.geeksforgeeks.org/height-and-depth-of-a-node-in-a-binary-tree/>

```
1 public int height(Node node) {
2     if (node == null) {
3         return 0;
4     }
5     int hleft = height(node.left);
6     int hright = height(node.right);
7     return Math.max(hleft, hright) + 1;
8 }
```

2. Đếm số lá trên cây nhị phân - [Count Leaves in Binary Tree](#)

– Ref - <https://www.geeksforgeeks.org/write-a-c-program-to-get-count-of-leaf-nodes-in-a-binary-tree/>

– Duyệt, kiểm tra 1 nút có là lá và đưa vào kết quả.

```
1 int countLeaves(Node root) {
2     if (root == null) {
3         return 0;
4     }
5     if (root.left == null && root.right == null) {
6         return 1;
7     }
8     return countLeaves(root.left) + countLeaves(root.right);
9 }
```

3. Số phần tử của cây nhị phân - [Size of Binary Tree](#)

– Ref - <https://www.geeksforgeeks.org/write-a-c-program-to-calculate-size-of-a-tree/>

```

1 int getSize(Node root) {
2     if (root == null)
3         return 0;
4     return getSize(root.left) + getSize(root.right) + 1;
5 }

```

4. Độ sâu tối thiểu của cây nhị phân - [Minimum Depth of a Binary Tree](#)

- Là nút lá có mức thấp nhất.
- Triển khai thuật toán duyệt cây và trong khi duyệt cây kiểm tra một nút có là nút lá và sau đó dựa vào mức của nút lá đó cập nhật kết quả.

```

1 int minDepth(Node root)
2 {
3     if (root == null) {
4         return 0;
5     }
6
7     Queue<Node> queue = new LinkedList<>();
8     queue.offer(root);
9     int depth = 1;
10
11     while (!queue.isEmpty()) {
12         int levelSize = queue.size();
13         for (int i = 0; i < levelSize; i++) {
14             Node current = queue.poll();
15             if (current.left == null && current.right == null) {
16                 return depth;
17             }
18
19             if (current.left != null) {
20                 queue.offer(current.left);
21             }
22             if (current.right != null) {
23                 queue.offer(current.right);
24             }
25         }
26         depth++;
27     }
28     return depth;
29 }

```

5. Liệt kê các phần tử mà không có họ hàng - [Print all nodes that don't have sibling](#)

– Ref - <https://www.geeksforgeeks.org/print-nodes-dont-sibling-binary-tree/>

6. Cây nhị phân hoàn chỉnh - [Complete binary tree](#)

- Ref - <https://www.geeksforgeeks.org/complete-binary-tree/>
- Duyệt và kiểm tra mỗi nút trong có đúng 2 con hay không.

7. Cây nhị phân hoàn hảo - [Perfect binary tree](#)

- Ref - <https://www.geeksforgeeks.org/perfect-binary-tree/>
- Kiểm tra mỗi nút trong có đúng 2 con.
- Kiểm tra các nút lá có cùng mức

8. Kiểm tra hai cây có trùng nhau - [Determine if Two Trees are Identical](#)

- Các thuật toán duyệt cây

1. Duyệt giữa - [Inorder Traversal](#)

- Duyệt giữa - [94. Binary Tree Inorder Traversal](#)
- Viết dưới dạng lặp

```

1 public static void inorder(Node root) {
2     if (root == null) {
3         return;
4     }
5
6     Stack<Node> stack = new Stack<>();
7     Node current = root;
8
9     while (current != null || !stack.isEmpty()) {
10        while (current != null) {
11            stack.push(current);
12            current = current.left;
13        }
14
15        current = stack.pop();
16        System.out.print(current.data + " ");
17
18        current = current.right;
19    }
20 }

```

- Viết dưới dạng đệ quy

```

1 public void inorder(Node root) {
2     if (root != null) {
3         inorder(root.left);
4         System.out.print(root.data + " ");
5         inorder(root.right);
6     }
7 }

```

2. Duyệt trước - [Preorder Traversal](#)

- Duyệt trước - [144. Binary Tree Preorder Traversal](#)

3. Duyệt sau - [Postorder Traversal](#)

- Duyệt sau - [145. Binary Tree Postorder Traversal](#)

4. Duyệt theo mức - [Level Order Traversal](#)

- Ref - <https://www.geeksforgeeks.org/level-order-tree-traversal/>

5. Duyệt theo mức ngược - [Reverse Level Order Traversal](#)

- Ref - <https://www.geeksforgeeks.org/reverse-level-order-traversal/>
- Sửa đổi thuật toán duyệt theo mức để ra được kết quả như yêu cầu đề bài.

6. Duyệt theo mức và hiển thị theo dòng - [Level order traversal Line by Line](#)

- Ref - <https://www.geeksforgeeks.org/level-order-traversal-line-line-set-3-using-one-queue/>
- Sửa đổi thuật toán duyệt theo mức để ra được kết quả như yêu cầu đề bài.

7. Các thuật toán xây dựng cây

- Xây dựng cây nhị phân hoàn toàn từ danh sách liên kết - [Make Binary Tree From Linked List](#)
 - Ref - <https://www.geeksforgeeks.org/given-linked-list-representation-of-complete-tree-convert-it-to-linked-representation/>
- Xây dựng cây nhị phân hoàn toàn từ kết quả duyệt theo mức - <https://www.geeksforgeeks.org/construct-complete-binary-tree-given-array/>
 - Ref - <https://www.geeksforgeeks.org/construct-complete-binary-tree-given-array/>
- Xây dựng cây từ phép duyệt giữa/duyet trước [Construct Tree from Inorder & Preorder](#)
 - Ref - <https://www.geeksforgeeks.org/construct-tree-from-given-inorder-and-preorder-traversal/>

- (d) Xây dựng cây đầy đủ từ phép duyệt trước và duyệt sau - Construct Full Binary Tree from given preorder and postorder traversals
 - Ref - <https://www.geeksforgeeks.org/full-and-complete-binary-tree-from-given-preorder-and-postorder-traversals/>

Tài liệu